

LAPORAN AKHIR
ANALISIS KOMPARATIF LATENSI CLOUD vs LOCALHOST
PADA PENGEMBANGAN GAME REACTION DUEL
BERBASIS WEBSOCKET

Disusun Oleh:

1.	Muhammad Ridho Jan Muhani	2410010567
2.	Muhammad Fairuz Nadhir Amrullah	2410010083
3.	Muhammad Adam Yazid Bustami	2410010113
4.	Muhammad Daffa Fadillah	2410010114

Mata Kuliah : Cloud Computing
Dosen Pengampu : Fathul Hafidh, S.Kom.

UNIVERSITAS ISLAM KALIMANTAN
MUHAMMAD ARSYAD AL BANJARI
PROGRAM STUDI TEKNIK INFORMATIKA
Banjarbaru, 2025

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi web modern telah membuka peluang pengembangan aplikasi interaktif berbasis real-time yang semakin kompleks dan responsif. Di antara implementasi paling menuntut dari paradigma ini adalah pengembangan game multiplayer, yang mensyaratkan sinkronisasi data lintas klien dalam rentang waktu milidetik. Keterlambatan transmisi data sekecil apapun dapat memengaruhi pengalaman pengguna secara signifikan, terutama pada jenis permainan yang bergantung pada presisi waktu reaksi.

Dalam literatur sistem terdistribusi, latensi jaringan (network latency) didefinisikan sebagai total waktu yang dibutuhkan sebuah paket data untuk melakukan perjalanan pulang-pergi antara sumber dan tujuan. Pada aplikasi real-time berbasis protokol WebSocket, latensi yang tinggi berdampak langsung pada ketidakakuratan sinkronisasi state antara server dan klien. Penelitian oleh Claypool dan Claypool (2006) dalam bidang game networking menegaskan bahwa untuk genre permainan aksi atau refleks, ambang batas toleransi latensi yang dapat diterima oleh pemain umumnya berada di bawah 100 milidetik; di atas nilai tersebut, persepsi terhadap ketidakadilan permainan meningkat secara signifikan.

Infrastruktur Platform as a Service (PaaS) pada layanan awan publik tingkat gratis (free-tier) menawarkan kemudahan penggelaran (deployment) namun seringkali memiliki keterbatasan pada throughput jaringan dan waktu respons server. Hal ini menjadi perhatian kritis ketika infrastruktur tersebut digunakan untuk menangani lalu lintas WebSocket berkecepatan tinggi yang bersifat persisten dan bidireksional.

Reaction Duel adalah sebuah game pengujian refleks kognitif dan ketangkasan berbasis web multiplayer yang dikembangkan sebagai proyek implementasi untuk mata kuliah Cloud Computing. Proyek ini awalnya dirancang dengan target penggelaran pada infrastruktur awan terdistribusi menggunakan platform Railway. Namun, dalam proses pengujian performa yang intensif, ditemukan kendala latensi jaringan yang signifikan, khususnya yang menyebabkan keterlambatan visual pada fitur Spectator Mode (mode penonton). Mengingat sifat permainan berbasis refleks yang menuntut presisi temporal hingga tingkat milidetik, dilakukan evaluasi ulang terhadap infrastruktur yang digunakan.

Sebagai respons terhadap temuan tersebut, proyek ini dialihkan ke infrastruktur Local Area Network (localhost). Keputusan ini secara akademis dapat dibingkai sebagai simulasi arsitektur Edge Computing, yaitu paradigma komputasi yang menempatkan pemrosesan data sedekat mungkin dengan sumber data guna meminimalkan latensi jaringan. Pemusatan server WebSocket dan basis data MySQL pada jaringan lokal terbukti mampu menekan latensi transmisi data mendekati 0 milidetik, sehingga integritas kompetisi berbasis waktu reaksi dapat terjaga sepenuhnya.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, penelitian ini merumuskan permasalahan sebagai berikut:

1. Bagaimana merancang arsitektur sistem game real-time yang responsif menggunakan protokol WebSocket (PHP Ratchet) dan apakah terdapat perbedaan performa latensi yang terukur antara infrastruktur PaaS cloud publik dengan lingkungan jaringan lokal (localhost)?
2. Bagaimana mengimplementasikan mekanik permainan yang dinamis beserta mekanisme validasi server-side (anti-cheat) untuk mencegah manipulasi data dari sisi klien?
3. Bagaimana mengelola dan memvisualisasikan data performa pemain secara sistematis guna menganalisis fenomena psikologi kognitif, yaitu Learning Effect dan Fatigue Effect?

1.3 Tujuan

Tujuan yang ingin dicapai melalui pengembangan proyek ini adalah:

4. Membangun game multiplayer real-time berbasis arsitektur client-server lokal yang berfungsi penuh dengan latensi minimal.
5. Mengimplementasikan sinkronisasi state permainan dan fitur Spectator Mode melalui protokol WebSocket dengan server PHP Ratchet sebagai Authoritative Server.
6. Menyediakan mekanik gameplay yang kaya, mencakup mode PvP (Player vs Player), mode VS AI, sistem skor berbasis Combo Multiplier, dan Global Live Chat.
7. Menyimpan data performa pemain secara terstruktur ke dalam basis data MySQL dan memvisualisasikannya ke dalam dasbor analitik interaktif menggunakan Chart.js.
8. Melakukan evaluasi komparatif terhadap infrastruktur cloud PaaS dan localhost untuk mengidentifikasi batasan-batasan teknis layanan cloud gratis pada kasus penggunaan WebSocket berkecepatan tinggi.

1.4 Batasan Masalah

Untuk menjaga fokus penelitian, beberapa batasan masalah ditetapkan sebagai berikut:

- Pengujian performa latensi dilakukan secara empiris melalui observasi perilaku sistem dan tidak mencakup pengukuran latensi jaringan secara terprogram dengan alat ukur khusus (network profiler).

- Implementasi mode VS AI menggunakan logika berbasis aturan (rule-based) dan bukan kecerdasan buatan berbasis pembelajaran mesin (machine learning).
- Lingkup pengujian sistem dibatasi pada jaringan lokal (LAN) dengan jumlah pengguna terbatas sesuai kapasitas pengujian tim pengembang.
- Analisis psikologi kognitif (Learning Effect dan Fatigue Effect) didasarkan pada data empiris yang dikumpulkan dari sesi permainan tim selama fase pengujian.

1.5 Pembagian Tugas Tim

No.	Nama Anggota	NPM	Tanggung Jawab
1	Muhammad Daffa Fadillah	2410010114	Backend Architecture & Backend Developer (PHP Ratchet, Game Loop, Anti-Cheat, Matchmaking)
2	Muhammad Fairuz Nadhir Amrullah	2410010083	Frontend Developer (UI/UX, Tailwind CSS, WebSocket Client)
3	Muhammad Adam Yazid Bustami	2410010113	Database Developer & Data Engineer (perancangan struktur database & skema)
4	Muhammad Ridho Jan Muhani	2410010567	Data Analyst (Dashboard Analitik, menganalisis data, mencari pola)

BAB II

TINJAUAN PUSTAKA

2.1 Arsitektur Client-Server dan Protokol WebSocket

Arsitektur client-server merupakan paradigma komputasi terdistribusi di mana dua atau lebih entitas perangkat lunak yang berbeda peran—klien (peminta layanan) dan server (penyedia layanan)—berkomunikasi melalui suatu jaringan. Dalam konteks game multiplayer berbasis web, server bertindak sebagai Authoritative Server, yaitu entitas tunggal yang memiliki otoritas penuh atas validasi aksi pemain dan pengelolaan state permainan. Pendekatan ini mencegah manipulasi data dari sisi klien karena seluruh logika krusial dieksekusi dan divalidasi di server.

Protokol HTTP konvensional bersifat stateless dan half-duplex, sehingga tidak efisien untuk komunikasi real-time yang memerlukan pengiriman data berkelanjutan dari server ke klien. Protokol WebSocket (RFC 6455) hadir sebagai solusi dengan menyediakan saluran komunikasi full-duplex yang persisten melalui koneksi TCP tunggal. Setelah proses handshake awal melalui HTTP (HTTP Upgrade), koneksi WebSocket tetap terbuka, memungkinkan server dan klien saling mengirim pesan kapan saja dengan overhead yang jauh lebih rendah dibandingkan polling HTTP berulang.

PHP Ratchet adalah pustaka (library) PHP yang mengimplementasikan protokol WebSocket di sisi server berdasarkan spesifikasi RFC 6455. Ratchet memanfaatkan ReactPHP sebagai fondasi event-loop berbasis I/O asinkron non-blocking, yang memungkinkan satu proses PHP tunggal menangani ribuan koneksi klien secara bersamaan tanpa pemblokiran (blocking). Dalam proyek ini, PHP Ratchet dipilih karena ekosistemnya yang matang dan kompatibilitasnya dengan lingkungan server LAMP (Linux, Apache, MySQL, PHP).

2.2 Edge Computing sebagai Respons terhadap Keterbatasan PaaS Cloud

Cloud Computing dalam model Platform as a Service (PaaS) menyediakan lingkungan pengembangan dan penggelaran aplikasi yang dikelola oleh penyedia cloud. Layanan ini menawarkan skalabilitas dan kemudahan manajemen infrastruktur. Namun, pada tingkat layanan gratis (free-tier), penyedia PaaS umumnya memberlakukan pembatasan pada sumber daya komputasi, bandwidth jaringan, dan waktu aktif server (uptime), yang berdampak pada peningkatan latensi, terutama untuk koneksi WebSocket yang bersifat persisten.

Edge Computing adalah paradigma komputasi terdistribusi yang memindahkan pemrosesan data dan logika aplikasi sedekat mungkin ke sumber data (pengguna akhir), alih-alih mengandalkan pusat data yang terpusat (cloud). Tujuan utamanya adalah meminimalkan latensi jaringan dengan mengeliminasi atau mempersingkat jalur perjalanan data. Dalam konteks proyek ini, penggelaran server WebSocket dan basis data pada jaringan lokal (localhost) dapat dipandang sebagai implementasi prinsip Edge Computing

pada skala yang kecil: server berada di lokasi yang sama dengan pengguna (zero-hop latency), sehingga latensi transmisi data mendekati nol.

2.3 Psikologi Kognitif: Learning Effect dan Fatigue Effect

Waktu reaksi (reaction time) adalah interval temporal antara kemunculan stimulus dan respons motorik individu terhadap stimulus tersebut. Variabel ini telah lama menjadi objek studi dalam psikologi eksperimental sebagai indikator performa kognitif. Dua fenomena utama yang konsisten ditemukan dalam penelitian performa manusia berulang adalah Learning Effect dan Fatigue Effect.

Learning Effect (atau Practice Effect) merujuk pada peningkatan performa yang terjadi akibat pengulangan suatu tugas. Dalam konteks game refleksi, pemain yang baru mengenal mekanik permainan akan mengalami penurunan waktu reaksi secara progresif pada sesi-sesi awal seiring terbentuknya memori motorik (motor memory) dan optimalisasi pola respons kognitif.

Fatigue Effect merujuk pada degradasi performa kognitif akibat kelelahan fisik atau mental yang terakumulasi. Setelah sejumlah sesi permainan berturut-turut, konsentrasi dan koordinasi motorik pemain akan menurun, yang termanifestasi sebagai peningkatan waktu reaksi rata-rata dan penurunan konsistensi. Proyek ini memanfaatkan data historis permainan untuk memvisualisasikan dan mengkonfirmasi kedua fenomena tersebut secara empiris.

BAB III

ANALISIS DAN PERANCANGAN SISTEM

3.1 Arsitektur Sistem

Sistem mengadopsi arsitektur client-server tiga lapis (three-tier architecture) yang berjalan dalam lingkungan jaringan lokal, terdiri dari:

- Client (Frontend): Antarmuka web dinamis yang dibangun menggunakan HTML5, JavaScript, dan kerangka kerja CSS Tailwind. Lapisan ini bertanggung jawab atas rendering tampilan game, penanganan input pengguna, dan manajemen koneksi WebSocket dari sisi klien.
- Server (Backend): Mesin peladen berbasis PHP yang mengintegrasikan PHP Ratchet untuk penanganan WebSocket. Server bertindak sebagai Authoritative Server—wasit tunggal yang mengelola seluruh siklus permainan (game loop), memvalidasi aksi pemain, dan mendistribusikan pembaruan state kepada seluruh klien yang terhubung.
- Database: Sistem manajemen basis data MySQL yang berfungsi sebagai repositori terpusat untuk persistensi data, meliputi profil pengguna, riwayat pertandingan, log obrolan, dan status kamar.

Diagram arsitektur berikut memvisualisasikan alur komunikasi antar komponen sistem:

[MASUKKAN DIAGRAM ARSITEKTUR DI SINI Buat diagram dengan kotak dan panah: Browser/Klien (HTML+JS) ↔ PHP Ratchet WebSocket Server ↔ MySQL Database. Gunakan draw.io atau SmartArt di Word.]

3.2 Perancangan Basis Data

Sistem menggunakan pendekatan basis data relasional (MySQL) yang terdiri dari empat entitas utama. Desain skema basis data dirancang untuk mendukung kebutuhan fungsional sistem, termasuk autentikasi pengguna, matchmaking, persistensi riwayat pertandingan, dan sistem obrolan global.

Tabel 3.1 Tabel users

Nama Kolom	Tipe Data	Keterangan / Fungsi
id	INT (PK, AI)	Identifier unik untuk setiap pengguna (Primary Key, Auto Increment)
username	VARCHAR(50)	Nama pengguna yang ditampilkan dalam permainan (unik)
password	VARCHAR(255)	Kata sandi yang telah di-hash menggunakan bcrypt untuk keamanan
xp	INT DEFAULT 0	Akumulasi Experience Points sebagai indikator kemajuan akun
level	INT DEFAULT 1	Level akun yang dikalkulasi berdasarkan nilai XP yang diperoleh
best_time	FLOAT NULL	Rekam jejak waktu reaksi terbaik pemain (dalam milidetik)
created_at	DATETIME	Stempel waktu pembuatan akun menggunakan DATETIME untuk kompatibilitas MySQL 5.7

Tabel 3.2 Tabel history

Nama Kolom	Tipe Data	Keterangan / Fungsi
id	INT (PK, AI)	Identifier unik untuk setiap catatan sesi pertandingan
user_id	INT (FK)	Kunci asing yang merujuk ke kolom id pada tabel users (relasi one-to-many)
final_score	INT	Skor akhir yang diperoleh pemain pada akhir sesi pertandingan
avg_reaction_time	FLOAT	Rata-rata waktu reaksi pemain dalam satu sesi (milidetik)
best_time_session	FLOAT	Waktu reaksi terbaik dalam sesi tersebut (milidetik)
game_mode	VARCHAR(20)	Mode permainan yang dimainkan: 'pvp', 'vs_ai', atau 'spectator'
played_at	DATETIME	Stempel waktu sesi permainan berlangsung

Tabel 3.3 Tabel rooms

Nama Kolom	Tipe Data	Keterangan / Fungsi
id	INT (PK, AI)	Identifier unik untuk setiap kamar permainan
room_code	VARCHAR(10)	Kode kamar unik yang digunakan untuk proses matchmaking
p1_id	INT (FK)	ID pengguna untuk pemain pertama (Player 1)
p2_id	INT (FK, NULL)	ID pengguna untuk pemain kedua; bernilai NULL saat kamar masih menunggu
p1_ready	TINYINT(1)	Flag kesiapan Pemain 1: 0 = belum siap, 1 = siap bermain
p2_ready	TINYINT(1)	Flag kesiapan Pemain 2: 0 = belum siap, 1 = siap bermain
status	ENUM	Status kamar saat ini: 'waiting' (menunggu lawan) atau 'playing'
created_at	DATETIME	Stempel waktu pembuatan kamar permainan

Tabel 3.4 Tabel chats

Nama Kolom	Tipe Data	Keterangan / Fungsi
id	INT (PK, AI)	Identifier unik untuk setiap pesan obrolan
user_id	INT (FK)	ID pengguna pengirim pesan; kunci asing ke tabel users
username	VARCHAR(50)	Nama pengguna pengirim (dicatat redundan untuk efisiensi query)
message	TEXT	Konten pesan teks yang dikirimkan pengguna
sent_at	DATETIME	Stempel waktu pengiriman pesan dalam format DATETIME

3.3 Perancangan Alur Permainan

Alur permainan (game flow) untuk satu sesi PvP (Player vs Player) berlangsung melalui tahapan-tahapan berikut secara sekuensial:

9. Autentikasi: Pemain melakukan login atau registrasi. Server memvalidasi kredensial terhadap basis data dan mengeluarkan session token.

10. Matchmaking: Pemain membuat atau bergabung ke dalam sebuah kamar berdasarkan kode unik. Status kamar diperbarui di basis data dari 'waiting' menjadi 'playing' setelah kedua pemain menyatakan kesiapan.
11. Sesi Permainan: Server memulai game loop, mengirimkan sinyal spawn objek target kepada seluruh klien yang terhubung (pemain dan penonton) secara bersamaan melalui broadcast WebSocket.
12. Kalkulasi Skor: Setiap klik pemain dikirim ke server sebagai event. Server memvalidasi waktu klik, menghitung skor berdasarkan jenis objek dan status kombo, lalu mengirimkan pembaruan scoreboard kepada semua klien.
13. Akhir Pertandingan: Setelah semua ronde selesai, server menghitung skor akhir, menentukan pemenang, dan menyimpan data riwayat sesi ke tabel history. XP pemain diperbarui di tabel users.

BAB IV

IMPLEMENTASI DAN PENGUJIAN

4.1 Arsitektur Backend: PHP Ratchet sebagai Authoritative Server

Server WebSocket dibangun di atas PHP Ratchet yang berjalan sebagai proses independen. Sebagai Authoritative Server, seluruh logika krusial permainan—termasuk penentuan waktu spawn objek, validasi klik pemain, dan kalkulasi skor—dieksekusi secara eksklusif di server. Klien tidak memiliki kemampuan untuk memodifikasi state permainan secara sepihak; klien hanya berwenang mengirimkan sinyal aksi (misalnya, event klik) dan menerima pembaruan state dari server.

Setiap koneksi klien diidentifikasi oleh server menggunakan Connection ID unik yang diberikan oleh Ratchet. Server memelihara daftar koneksi aktif yang terbagi dalam kelompok-kelompok kamar (rooms), sehingga broadcast pesan dapat ditargetkan hanya kepada klien yang berada dalam kamar yang sama, bukan kepada seluruh koneksi global. Pendekatan ini mengoptimalkan penggunaan bandwidth dan mengurangi pemrosesan yang tidak perlu.

4.2 Mekanik Permainan dan Sistem Skor

Sistem objek target permainan mengimplementasikan tiga jenis objek dengan karakteristik dan nilai yang berbeda, sebagai berikut:

Jenis Objek	Nilai Skor	Efek terhadap Kombo	Karakteristik Khusus
Objek Hijau (Target)	+100 poin	+1 indeks Combo Multiplier (maks. 10x)	Objek utama; poin efektif dikalikan dengan multiplier kombo aktif
Berlian Emas (Bonus)	+250 poin	Tidak memengaruhi kombo	Durasi kemunculan sangat singkat (despawn cepat); muncul acak
Bom (Rintangan)	-50 poin	Mereset Combo Multiplier ke 1x	Penalti ganda: pengurangan skor dan hilangnya akumulasi kombo

Sistem Combo Multiplier dirancang untuk memberikan penghargaan kepada pemain yang mampu mempertahankan akurasi klik secara konsisten. Setiap klik berhasil pada Objek Hijau menaikkan indeks multiplier sebesar 1, sehingga pada kombo ke-10, setiap klik efektif bernilai $10 \times 100 = 1.000$ poin. Sebaliknya, satu klik pada Bom mereset seluruh akumulasi kombo, menciptakan dinamika risiko-imbalan (risk-reward) yang strategis.

4.3 Mekanisme Anti-Cheat

Sistem anti-cheat diimplementasikan melalui beberapa mekanisme komplementer yang bekerja di sisi server:

a) Penguncian Layar Sebelum Aba-Aba: Pada fase countdown sebelum pertandingan dimulai, server menonaktifkan secara eksplisit kemampuan klien untuk mendaftarkan klik yang valid. Segala aksi klik yang dikirim dalam periode ini dikategorikan sebagai FALSE_START dan diabaikan oleh sistem skor server.

b) Sinkronisasi Waktu dengan UTC_TIMESTAMP(): Pencatatan seluruh stempel waktu (timestamp) kritis—termasuk waktu spawn objek dan waktu klik pemain—dilakukan menggunakan fungsi UTC_TIMESTAMP() pada sisi server MySQL. Pendekatan ini mengeliminasi risiko manipulasi waktu dari sisi klien, karena waktu yang digunakan sebagai acuan kalkulasi adalah waktu server yang tidak dapat dimodifikasi oleh pemain.

c) Validasi Klik Dini (TOO_EARLY Detection): Jika server menerima event klik dari klien sebelum sinyal spawn objek resmi dikirimkan, event tersebut diklasifikasikan sebagai TOO_EARLY. Server tidak hanya mengabaikan aksi tersebut, tetapi juga memberikan penalti waktu kepada pemain yang bersangkutan, sehingga menciptakan disinsentif aktif terhadap perilaku curang.

Keseluruhan mekanisme ini memastikan bahwa kalkulasi waktu reaksi yang ditampilkan mencerminkan performa kognitif aktual pemain, bukan artefak dari manipulasi klien.

4.4 Sinkronisasi Real-Time dan Spectator Mode

Sinkronisasi state permainan dikelola melalui sistem broadcast WebSocket yang terstruktur. Host permainan (pemain yang pertama kali membuat kamar) memiliki peran tambahan sebagai pengirim sinyal transisi ronde (round_end). Setelah server memvalidasi berakhirnya sebuah ronde, sinyal transisi dikirimkan secara bersamaan (simultaneous broadcast) kepada seluruh koneksi yang terdaftar dalam kamar tersebut, termasuk penonton.

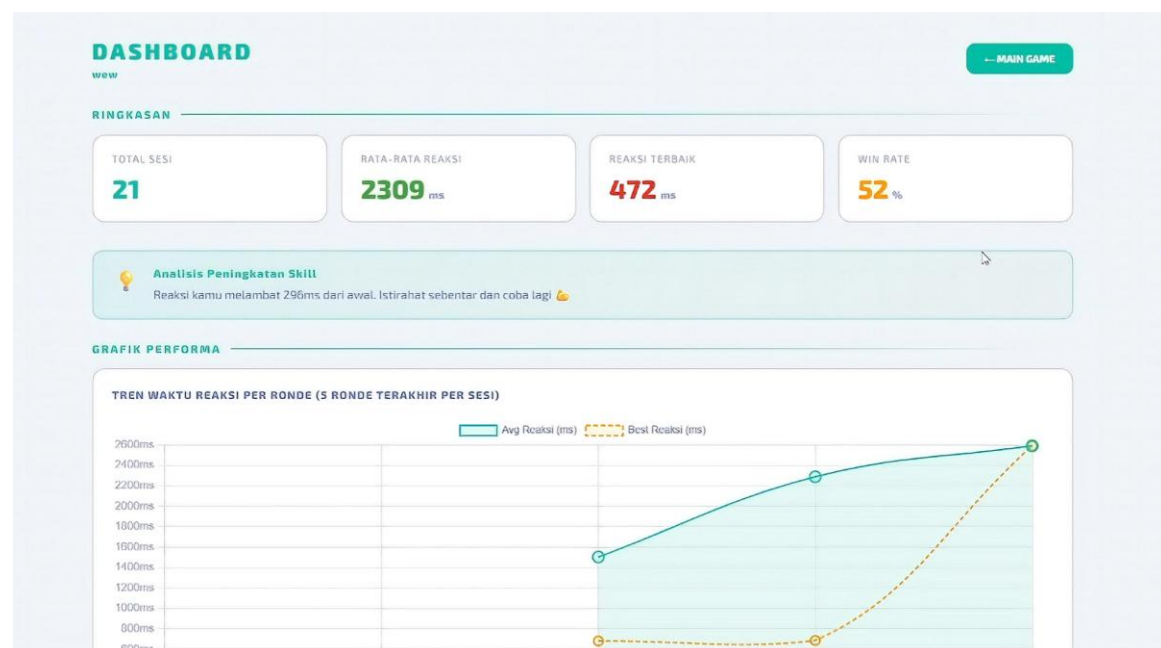
Spectator Mode memungkinkan pengguna ketiga untuk mengamati pertandingan secara langsung tanpa memengaruhi jalannya permainan. Implementasi awal Spectator Mode menghadapi kendala desinkronisasi visual—penonton seringkali tertinggal satu ronde di belakang para pemain aktif. Permasalahan ini diselesaikan dengan arsitektur broadcast yang lebih ketat: server tidak membedakan klien pemain dan klien penonton dalam hal distribusi pembaruan state; semua klien menerima paket data yang identik pada saat yang sama. Perbedaan hanya terletak pada cara klien merender data tersebut berdasarkan peran yang ditetapkan saat koneksi.

4.5 Dasbor Statistik dan Analisis Performa

Data yang tersimpan dalam tabel history diekstrak melalui kueri SQL dan disajikan secara visual pada halaman Dasbor menggunakan pustaka Chart.js. Dasbor menyajikan dua jenis visualisasi utama:

- **Grafik Garis (Line Chart):** Menampilkan tren rata-rata waktu reaksi pemain dari sesi ke sesi secara kronologis. Grafik ini dirancang untuk memvisualisasikan kurva pembelajaran pemain dari waktu ke waktu.
- **Grafik Batang (Bar Chart):** Menampilkan distribusi skor per sesi, yang berguna untuk mengidentifikasi sesi dengan performa puncak atau sesi dengan performa di bawah rata-rata.

Dasbor juga dilengkapi dengan algoritma evaluasi dinamis yang membandingkan rata-rata performa lima sesi pertama dengan rata-rata performa lima sesi terakhir. Perbedaan statistik antara dua kelompok data ini digunakan untuk mendeteksi dan menampilkan indikator Learning Curve secara otomatis.



Berdasarkan data empiris yang dikumpulkan dari puluhan sesi pengujian, sistem memvisualisasikan dua pola performa yang konsisten:

- **Learning Effect:** Pada 5 hingga 10 sesi pertama, grafik waktu reaksi menunjukkan penurunan yang signifikan—refleks pemain menjadi semakin tajam seiring terbentuknya adaptasi motorik terhadap mekanik permainan.
- **Fatigue Effect:** Setelah melewati 20 sesi berturut-turut, grafik konsistensi memburuk dan waktu reaksi rata-rata kembali meningkat, yang membuktikan bahwa kelelahan fisik menurunkan kapasitas fokus dan respons motorik pemain.

4.6 Optimalisasi Sistem (Quality Assurance)

Melalui proses Quality Assurance (QA) yang sistematis, beberapa optimalisasi kritis berhasil diidentifikasi dan diterapkan:

a) Delta Fetching: Pada implementasi awal, mekanisme pembaruan log event meminta klien untuk mengambil (fetch) seluruh riwayat event dari server secara berulang, yang menyebabkan payload data membengkak seiring berjalannya permainan. Masalah ini diatasi dengan menerapkan prinsip Delta Fetching: server hanya mengirimkan data event yang belum diterima oleh klien (delta), bukan keseluruhan riwayat. Optimalisasi ini secara signifikan mengurangi beban jaringan dan eliminasi lag yang terjadi pada fase akhir pertandingan.

b) Responsivitas Perangkat Seluler: Antarmuka web dioptimalkan untuk perangkat seluler melalui dua pendekatan. Pertama, penundaan klik bawaan peramban seluler (300ms touch delay) dieliminasi dengan menerapkan properti CSS touch-action: manipulation pada elemen-elemen interaktif. Kedua, kebijakan autoplay media (suara notifikasi game) pada peramban seluler yang membatasi pemutaran otomatis diatasi dengan menambahkan atribut playsinline pada elemen media HTML.

c) Kompatibilitas Basis Data: Dalam proses pengujian, ditemukan ketidakkompatibilan antara tipe data TIMESTAMP dengan lingkungan MySQL versi 5.7 yang digunakan dalam server pengembangan. Kolom-kolom yang menggunakan TIMESTAMP dengan nilai default CURRENT_TIMESTAMP() menghasilkan perilaku yang tidak konsisten. Masalah ini diselesaikan dengan migrasi tipe data ke DATETIME untuk seluruh kolom stempel waktu, yang memastikan kompatibilitas penuh dan perilaku yang konsisten pada MySQL 5.7.

4.7 Evaluasi Komparatif: Cloud PaaS vs Localhost

Evaluasi performa dilakukan melalui pengujian empiris pada dua skenario infrastruktur yang berbeda. Tabel berikut merangkum temuan utama dari evaluasi komparatif tersebut:

Parameter Evaluasi	Cloud PaaS (Railway Free-Tier)	Localhost (LAN)
Estimasi Latensi WebSocket	Tinggi (terukur secara visual: >100ms)	Sangat Rendah (mendekati 0ms)
Konsistensi Spektator Mode	Tidak konsisten (desinkronisasi visual antar-ronde)	Konsisten (sinkron sempurna)
Ketersediaan Layanan (Uptime)	Terbatas (pembatasan free-tier aktif)	100% selama sesi pengujian berlangsung
Kemudahan Penggelaran (Deploy)	Sangat mudah (otomatis via Git)	Manual (konfigurasi XAMPP/LAMP lokal)

Parameter Evaluasi	Cloud PaaS (Railway Free-Tier)	Localhost (LAN)
Biaya Infrastruktur	Gratis (dengan batasan sumber daya)	Gratis (memanfaatkan perangkat keras lokal)
Kecocokan untuk Game Refleks	Tidak memadai (latensi melebihi ambang toleransi)	Sangat memadai (latensi di bawah ambang toleransi)

Temuan ini mengkonfirmasi bahwa layanan cloud PaaS tingkat gratis memiliki keterbatasan fundamental yang membuatnya tidak cocok untuk aplikasi WebSocket berbasis refleks. Keputusan migrasi ke localhost, yang dalam perspektif akademis merepresentasikan penerapan prinsip Edge Computing, terbukti secara empiris sebagai satu-satunya pilihan yang mampu memenuhi persyaratan latensi proyek ini.

BAB V

PENUTUP

5.1 Kesimpulan

Proyek Reaction Duel telah berhasil diimplementasikan secara utuh sebagai aplikasi game multiplayer berbasis web dengan seluruh fitur inti yang direncanakan. Berikut adalah kesimpulan yang dapat ditarik berdasarkan proses pengembangan dan pengujian yang telah dilaksanakan:

14. Arsitektur Client-Server dengan PHP Ratchet: Sistem WebSocket berbasis PHP Ratchet berhasil diimplementasikan sebagai Authoritative Server yang andal. Server mampu mengelola siklus permainan (game loop), mendistribusikan pembaruan state secara real-time, dan mempertahankan integritas permainan melalui mekanisme validasi server-side.
15. Evaluasi Infrastruktur Cloud vs Localhost: Pengujian komparatif mengkonfirmasi secara empiris bahwa infrastruktur PaaS cloud publik tingkat gratis tidak memadai untuk menangani lalu lintas WebSocket berkecepatan tinggi yang dibutuhkan oleh game berbasis refleks. Latensi yang terlalu tinggi pada platform Railway mengakibatkan desinkronisasi visual yang merusak integritas kompetisi. Migrasi ke lingkungan localhost—yang dalam kerangka akademis merepresentasikan simulasi prinsip Edge Computing—terbukti efektif mengeliminasi kendala latensi dan memulihkan presisi temporal sistem.
16. Validasi Psikologi Kognitif: Data empiris yang dikumpulkan dari puluhan sesi pengujian dan divisualisasikan melalui Dasbor Chart.js berhasil mengkonfirmasi keberadaan Learning Effect (peningkatan performa pada sesi-sesi awal) dan Fatigue Effect (degradasi performa setelah penggunaan berkepanjangan). Temuan ini memvalidasi Reaction Duel sebagai instrumen yang sah untuk mengukur variabel waktu reaksi kognitif dalam konteks penelitian perilaku.
17. Kelengkapan Fitur Sistem: Seluruh fitur yang direncanakan berhasil diimplementasikan, meliputi matchmaking dinamis, mode VS AI berbasis aturan, sistem Combo Multiplier, Spectator Mode yang tersinkronisasi, Global Live Chat, serta dasbor analitik performa berbasis Chart.js.

5.2 Saran

Berdasarkan pengalaman pengembangan dan keterbatasan yang ditemui, beberapa saran diberikan untuk pengembangan lebih lanjut:

18. Migrasi ke Infrastruktur Cloud Berbayar atau Self-Hosted VPS: Untuk penyelenggaraan versi publik Reaction Duel, disarankan untuk menggunakan server Virtual Private Server (VPS) yang dikelola sendiri atau layanan cloud berbayar dengan jaminan Service Level Agreement (SLA) untuk latensi jaringan, guna mendapatkan keseimbangan antara aksesibilitas dan performa.

19. Pengukuran Latensi Terprogram: Penelitian lanjutan dapat mengintegrasikan alat pengukuran latensi WebSocket yang terprogram (misalnya, mekanisme ping-pong interval) untuk menghasilkan data kuantitatif yang lebih presisi dalam evaluasi komparatif infrastruktur.
20. Pengembangan Kecerdasan Buatan yang Lebih Adaptif: Mode VS AI dapat ditingkatkan dengan menerapkan algoritma kecerdasan buatan yang lebih canggih, misalnya model berbasis reinforcement learning yang dapat beradaptasi dengan tingkat kemampuan pemain secara dinamis.
21. Perluasan Basis Pengujian: Validitas analisis psikologi kognitif (Learning Effect dan Fatigue Effect) dapat ditingkatkan secara signifikan dengan memperluas basis data pengujian melalui keterlibatan lebih banyak subjek uji dengan profil demografis yang beragam.

5.3 Daftar Pustaka

- Claypool, M., & Claypool, K. (2006). Latency and Player Actions in Online Games. *Communications of the ACM*, 49(11), 40–45.
- Fette, I., & Melnikov, A. (2011). The WebSocket Protocol (RFC 6455). Internet Engineering Task Force (IETF). <https://datatracker.ietf.org/doc/html/rfc6455>
- Shi, W., Cao, J., Zhang, Q., Li, Y., & Xu, L. (2016). Edge Computing: Vision and Challenges. *IEEE Internet of Things Journal*, 3(5), 637–646.
- Ratchet: PHP WebSocket Library. (2024). Ratchet Documentation. <http://socketo.me>
- Chart.js Contributors. (2024). Chart.js Documentation. <https://www.chartjs.org/docs/>
- Mell, P., & Grance, T. (2011). The NIST Definition of Cloud Computing (Special Publication 800-145). National Institute of Standards and Technology.
- Schmidt, R. A., & Lee, T. D. (2011). *Motor Control and Learning: A Behavioral Emphasis* (5th ed.). Human Kinetics.